PPSITMAJM402 DEEP LEARNING

Ms. Sophia Dcruz

UNIT II: DEEP NETWORKS

INTRODUCTION

- A **Deep Neural Network (DNN)** is an **artificial neural network (ANN)** with **multiple layers between the input and output layers**
- There are different types of neural networks but they always consist of the same components:
  - Neurons
  - Synapses
  - Weights
  - Biases
  - Functions
- These components function similar to the **human brain**
- DNNs can be trained like any other **machine learning algorithm**
- Each mathematical manipulation is considered a **layer**
- Complex DNNs have **many layers**
- Hence, they are called **deep networks**

Deep Learning Overview

- Deep learning is part of a broader family of **machine learning methods**
- It is based on **artificial neural networks with representation learning**
- Learning can be:
  - Supervised
  - Semi-supervised
  - Unsupervised
- **Example of DNN**
- A DNN trained to **recognize dog breeds**
- The network:
  - Takes an image as input
  - Calculates probability that the dog belongs to a certain breed
- User can:
  - Review the results
  - Select which probabilities should be displayed (above a threshold)
- The proposed label is returned as output

Capabilities of DNNs

- DNNs can model **complex non-linear relationships**
- DNN architectures generate **compositional models**
- Objects are expressed as a **layered composition of primitives**
- Extra layers enable:
  - Composition of features from lower layers
  - Modeling complex data with fewer units than shallow networks

DEEP FEEDFORWARD NETWORK

- Multi-layered Network of neurons is composed of many **sigmoid neurons**
- MLNs can handle **non-linearly separable data**
- Layers between input and output layers are called **hidden layers**
- Hidden layers handle: Complex non-linear relations between input and output
- Goal of Feedforward Network is to approximate some function **f***
- For a classifier:
- $y = f^*(x)$maps input x to category y
- Learns parameters **θ** for best function approximation

- Simple neural networks solve **complex non-linear decision boundaries**
- **Example: Mobile Phone Like/Dislike Predictor**
- Input variables:
  - Screen size
  - Cost
- Decision boundary is complex and non-linear
- **Simple Neural Network Structure**
- Inputs:
  - x1 → Screen size
  - x2 → Price
- Bias:
  - b1, b2

First Neuron

- Connected to x1 and x2
- Weights:
  - w11, w12
- Output:
  - h1
- Function:
  - $h1 = f1(x' 1x2)$

Output of First Neuron

- Sigmoid function applied
- Output h1 is A real value between 0 and 1

Second Neuron

- Connected to x1 and x2
- Weights:
  - w13, w14
- Output:
  - h2
- Function:
  - $h2 = f2(x' 1x2)$

Output Neuron

- Takes h1 and h2 as input
- Produces final predicted output

Generic Deep Neural Network

- Network with:
  - Two hidden layers (can be more)
  - Each hidden layer has sigmoid neurons
- Three inputs
- Two output neurons
- **Neuron Operations**
- For each neuron:
- **Pre-activation (a)**: Weighted sum of inputs plus bias
- **Activation (h)**: Sigmoid function

Weight and Bias Notation

- W111: Weight of first neuron in first hidden layer connected to first input
- W112: Weight of first neuron connected to second input
- b11: Bias of first neuron in first hidden layer

Pre-activation Equation

- Pre-activation at layer i: $a_i(x) = W_i h_{i-1}(x) + b_i$

Activation Equation

- Activation at layer i: $h_i(x) = g\big(a_i(x)\big)$
- g is the **activation function (sigmoid)**

Predicted Output

- $f(x) = h_L = O(a_L)$
- O is the **output activation function**

Softmax Function

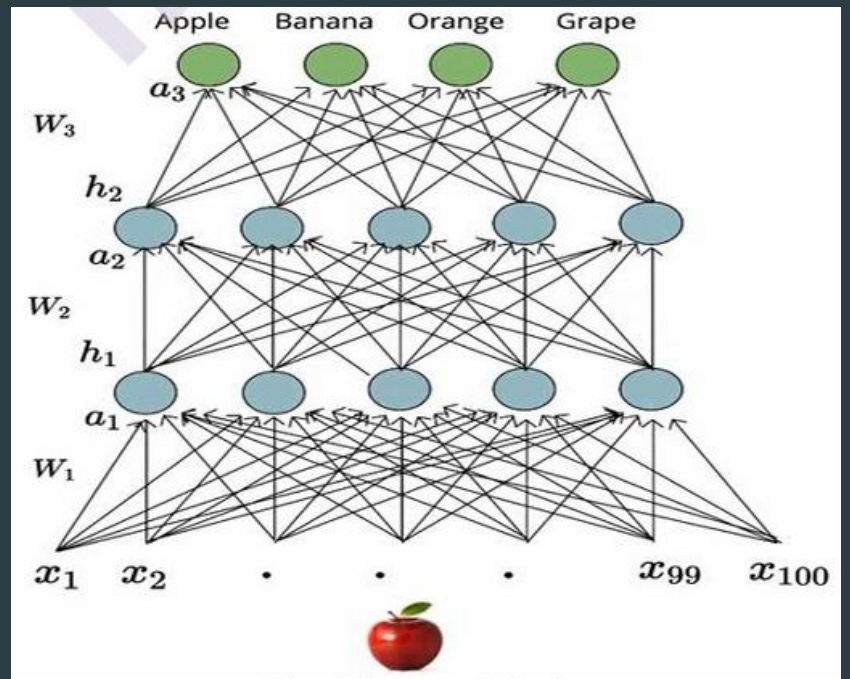


Fig. 6 Softmax Fubction.

- In the Softmax function, the output is **always positive**, irrespective of the input.
- Softmax converts raw scores into a **probability distribution.**
- The sum of all output probabilities is **equal to 1.**
- **Fig. 6:** Softmax Function

Softmax in Neural Networks

- Consider a network with **4 output neurons.**
- The output of these neurons is represented as a vector **a.**
- Softmax activation is applied to vector **a.**
- The output becomes a **predicted probability distribution.**

Loss Function

Purpose of Loss Function

- The purpose of the loss function is to tell the model that **correction is needed.**
- It guides the learning process during training.

Binary vs Multi-class Classification

- Number of output neurons usually equals the **number of classes.**
- For **binary classification:**
  - A single **sigmoid neuron** is used.
  - Outputs probability **P(Y = 1).**
  - **P(Y = 0) = 1 − P(Y = 1)**

Cross-Entropy Loss

- Cross-entropy loss compares:
  - Predicted probability distribution
  - True probability distribution
- Used for:
  - Binary classification
  - Multi-class classification

Gradient-Based Learning

- Training a neural network is similar to training other ML models using **gradient descent**.
- Main difference:
  - Neural networks are **non-linear**
  - Loss functions become **non-convex**

Optimization Difference

- Linear models:
  - Use linear equation solvers
  - Use convex optimization
- Neural networks:
  - Use **iterative gradient-based optimizers**
  - Drive the cost function to a **very low value**, not necessarily global minimum

Ball-in-Valley Analogy

- Gradient descent can be visualized as:
  - A ball dropped into a **deep valley**
  - The ball settles at the **bottom**
- Goal:
  - Minimize cost function to reach minimum
- **Fig. 7:** Ball diagram for gradient-based learning

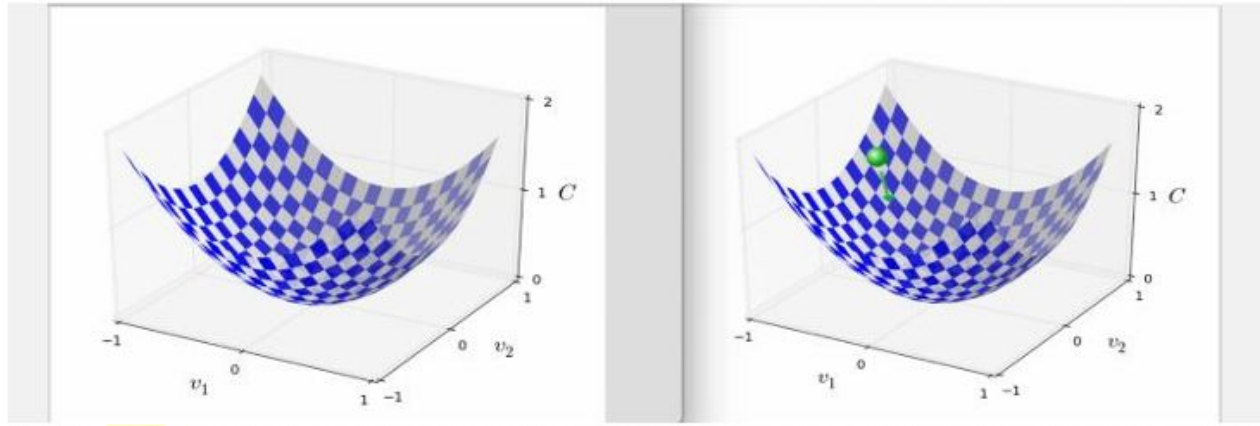


Fig 7. Ball diagram depicting the visualisation of how gradient based learning occur.

Gradient Representation

- Change in cost:
  - $\Delta C \approx \nabla C \cdot \Delta v$
- ∇C = Gradient of cost
- Δv = Change in parameters

Stochastic Gradient Descent

- When number of training inputs is very large training becomes **slow**
- To overcome this randomly selects a small number **m** of training samples
- These samples form a **mini-batch**
- Average gradient of mini-batch approximates full gradient
- Reduces **computational load**

Limitations of SGD

- No convergence guarantee for **non-convex loss functions**
- Sensitive to **initial parameter values**
- Weights should be:
  - Initialized to **small random values**
- Biases:
  - Zero or small positive values

Regularization for Deep Learning

- Regularization reduces **generalization error**
- Prevents **overfitting**
- Keeps training error as low as possible
- Regularization trades:
  - Increased bias
  - Reduced variance
- Effective regularizer:
  - Reduces variance significantly
  - Does not increase bias too much

Occam's Razor Principle

- Regularization leads to **simpler models**
- Simpler models generalize better
- Constraints reduce solution space

Bias-Variance Tradeoff

- Overfitting ↔ Underfitting
- Bias–variance tradeoff explains both
- **Fig. 9:** Underfitting and Overfitting

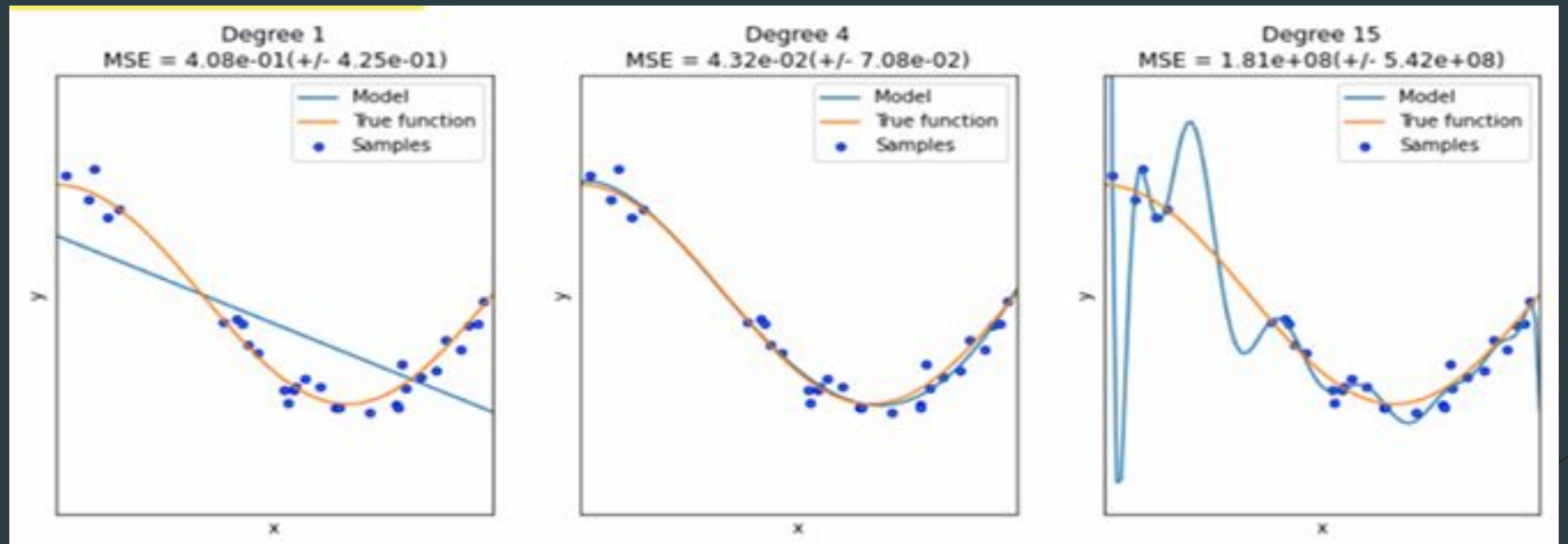


Fig. 9 Underfitting and overfitting

Bias

- Error due to **wrong assumptions**
- High bias → **Underfitting**
- Model misses relevant relationships

Variance

- Error due to **sensitivity to data fluctuations**
- High variance → **Overfitting**
- Model learns noise

Goal of Regularization

- Minimize both bias and variance
- Improve generalization

L2 Regularization

- Also known as: Weight decay or Ridge Regression
- Extends loss function with regularization term **Ω**
- **Regularization Term**= Sum of squared weights ($\lambda 1 \ ||w||^2$ + loss)

- **Effect of L2 Regularization**
- Penalizes large weights
- Reduces variance
- Makes generalization easier

- **Alpha (α)**
- Called **regularization rate**
- Controls strength of regularization

L1 Regularization

- Also known as **Lasso Regression**
- Regularization term: Sum of absolute weight values ($\lambda_1 ||w||_1$ + Loss)
- **Effect of L1 Regularization**
- Introduces **sparsity**
- Forces some weights to become **zero**
- Performs feature selection
- **Difference from L2**
- L2: Reduces magnitude of all weights
- L1: Eliminates some weights completely

Elastic Net Regularization

- Combines L1 and L2 regularization
- Penalty: $\lambda_1 ||w||_1 + \lambda_2 ||w||^2_2$
- Combines:
  - Feature elimination (L1)
  - Weight reduction (L2)

What Does Regularization Achieve?

- L2 encourages weights → **near zero**
- L1 encourages weights → **exact zero**
- Smaller weights reduce neuron impact
- Network complexity reduces
- Overfitting is avoided

Entropy Regularization

- Applied to **probabilistic models**
- Used in Reinforcement Learning
- Adds entropy penalty to loss
- Entropy measures uncertainty or information
- Reduces variance

In Reinforcement Learning

- Promotes **action diversity**
- Encourages exploration

Dropout

- Randomly turns off neurons during training
- Each neuron dropped with probability **p**
- Injects noise into training

Dropout During Testing

- All neurons are active
- Activations are scaled down
- Ensures consistency

Effect of Dropout

- Forces alternative paths
- Approximates training many networks
- Makes training noisy

Dropout Variations

- **Inverted Dropout:** Scales activations during training so that no scaling is needed at test time.
- **Gaussian Dropout:** Multiplies activations with Gaussian noise instead of randomly setting them to zero.
- **DropConnect:** Randomly drops weights (connections) instead of dropping neurons.
- **Variational Dropout:** Uses the same dropout mask across time steps or layers, commonly in RNNs.
- **Attention Dropout:** Applies dropout to attention weights to prevent over-reliance on specific tokens or regions.
- **Adaptive Dropout:** Adjusts dropout probability dynamically based on neuron importance or training progress.
- **Embedding Dropout:** Randomly drops entire word or feature embeddings during training.
- **DropBlock:** Drops contiguous regions of feature maps, mainly used in CNNs for stronger regularization.

Stochastic Depth

- Drops entire network blocks
- Used in deep ResNets
- Bypasses layers using skip connections
- Probability depends on depth

Early Stopping

- Stop training when:
  - Training error decreases
  - Validation error increases
- Restore best parameters

Parameter Sharing

- Forces parameters to be equal
- Uses domain knowledge
- Best example:
  - **Convolutional Neural Networks**
- Same kernel used across image

Batch Normalization

- Normalizes mean and variance
- Brings features to same range
- Acts as regularizer

Data Augmentation

- Generates new training samples
- Reduces variance
- Acts as noise injection

Types of Data Augmentation

- Basic transformations (flip, rotate, crop)
- Feature space augmentation
- GAN-based augmentation
- Meta-learning based augmentation

Types of Data Augmentation

- **Basic transformations (flip, rotate, crop):** Create new training samples by applying simple geometric changes to existing data.
- **Feature space augmentation:** Generates new samples by modifying or interpolating data in the learned feature space rather than raw input space.
- **GAN-based augmentation:** Uses Generative Adversarial Networks to synthesize realistic new training data.
- **Meta-learning based augmentation:** Learns optimal augmentation strategies automatically using meta-learning during training.

OPTIMIZATION FOR TRAINING DEEP MODELS

- Deep learning algorithms involve optimization in many contexts.
- For example, performing inference in models such as **PCA** involves solving an optimization problem.
- We often use **analytical optimization** to write proofs or design algorithms.
- Of all optimization problems in deep learning, the **most difficult is neural network training.**
- It is common to invest **days to months of time** on **hundreds of machines** to solve even a single instance of neural network training.
- Because this problem is so important and expensive, a **specialized set of optimization techniques** has been developed.
- **Example:**
  Training a large CNN like ResNet on ImageNet may take weeks on GPUs.

How Learning Differs from Pure Optimization

- Optimization algorithms for training deep models differ from traditional optimization algorithms.
- Machine learning usually acts **indirectly**.
- We care about a **performance measure P** defined with respect to the **test set**, which may be **intractable**.
- We therefore optimize a **different cost function J(θ)** in the hope that doing so improves **P**.
- In pure optimization, minimizing **J(θ)** is the goal itself.
- **Example:**
- P = Accuracy
- J(θ) = Cross-entropy loss

Generalization Error and Risk

- The expected generalization error (risk) is taken over the **true data-generating distribution p_data**.
- If p_data is known, it becomes a **pure optimization problem**.
- In practice, p_data is unknown and we have only a **finite training set**.
- This converts the problem into a **machine learning problem**.

Empirical Risk Minimization (ERM)

- The ML problem is converted back to optimization by replacing p_data with the **empirical distribution p̂_data**.
- The goal becomes minimizing **empirical risk**.
- This is called **Empirical Risk Minimization (ERM)**.
- The expectation is now taken over the **training data distribution**, not the true distribution.

Problems with ERM

- ERM is **prone to overfitting**, especially with **high-capacity models**.
- High-capacity models can **memorize the dataset**.
- ERM may not be feasible because most optimization algorithms use **Gradient Descent**, which requires **derivatives**.
- Some loss functions like **0-1 loss** are **non-differentiable**.

Surrogate Loss Functions (SLF)

- A **surrogate loss function** is used as a proxy.
- Example: **Negative log-likelihood / Cross-entropy** is used instead of **0–1 loss**.
- 0–1 loss is **non-differentiable**, so GD cannot be applied.
- Cross-entropy is a **smooth approximation** of the 0–1 loss.

Example:

- Spam vs Not Spam
- Instead of checking correct/incorrect (0–1), cross-entropy measures *how confident* the prediction is.

Benefits of Surrogate Loss Functions

- SLFs allow the model to **keep improving confidence**.
- With 0–1 loss, once prediction crosses a threshold (e.g., 0.5), no further improvement is encouraged.
- Cross-entropy pushes prediction scores closer to the true class.

Types of Gradient Descent

- **Batch Gradient Descent (BGD):** Uses entire dataset.
- **Stochastic Gradient Descent (SGD):** Uses one example.
- **Mini-batch Gradient Descent (MBGD):** Uses a small batch.
- Mini-batch GD is commonly referred to as **SGD** today.
- **Example:**
  Batch size = 32 or 64 (powers of 2 for hardware efficiency)

Effect of Mini-batches

- Small batches inject **noise**, acting as **regularization**.
- Noise helps escape local minima and saddle points.
-

Random Reshuffling

- Mini-batches should be selected **randomly**.
- Shuffling once per epoch is sufficient.
- First epoch gives an **unbiased gradient estimate**.
- Subsequent epochs produce **biased estimates**, but they work well in practice.
- This approach is called **Random Reshuffling**.
-

Local Minima

- Convex functions guarantee global minima.
- Neural networks have **many local minima**.
- Caused by **model identifiability problem**.
- Neural networks are **not identifiable** due to **weight space symmetry**.

Problematic Local Minima

- High-cost local minima are problematic.
- Many low-gradient points are **not minima**.
- Modern practice aims for **low cost**, not necessarily minimum cost.

Plateaus, Saddle Points, and Flat Regions

Saddle Points

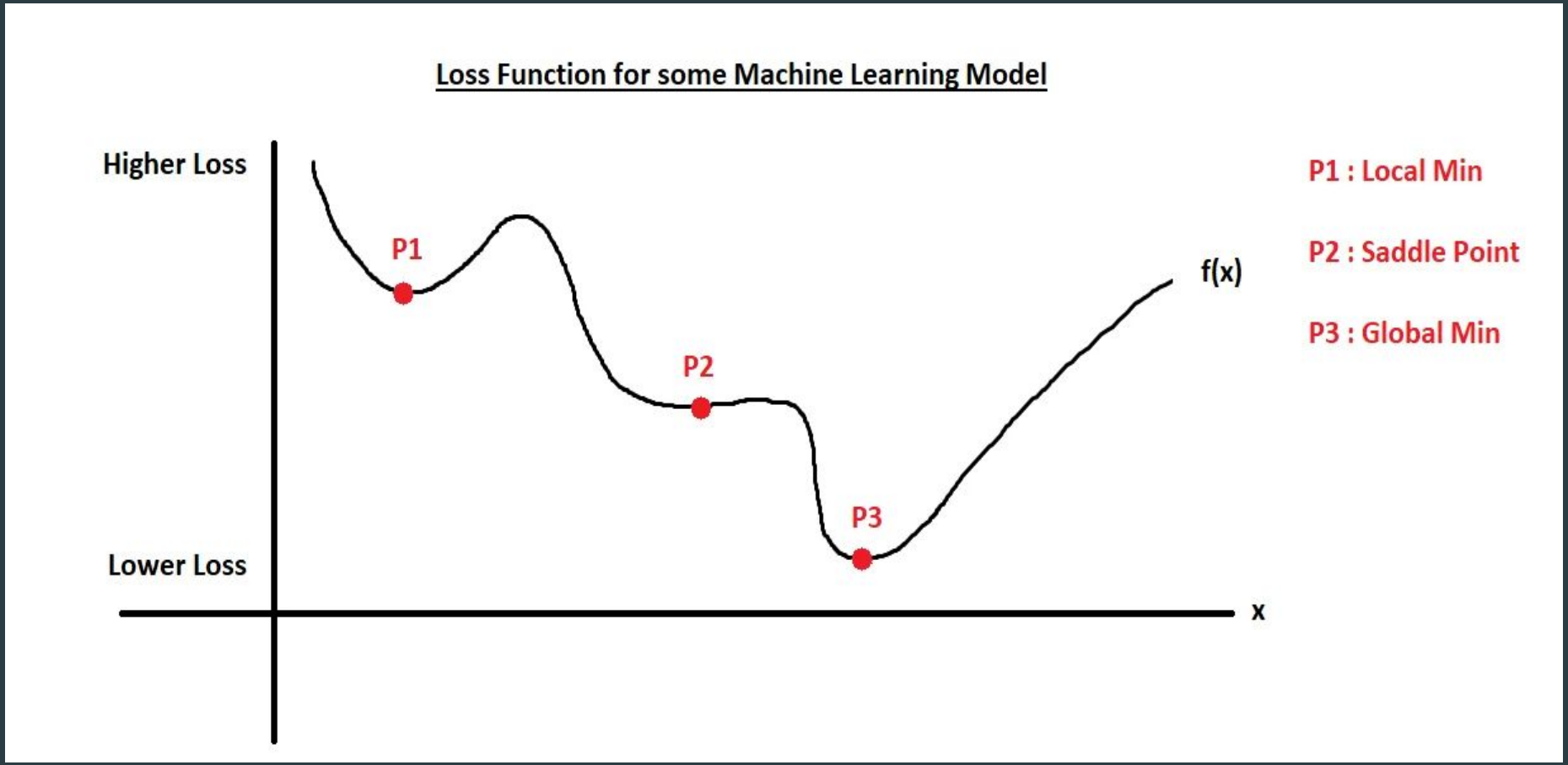
- Saddle points have **zero gradient**.
- More common in **high-dimensional spaces**.
- SGD tends to **avoid high-cost saddle points**.

Plateaus

- Plateaus are flat regions with **high cost**.
- Training can get stuck due to **near-zero gradients**.

Cliffs and Exploding Gradients

- Deep networks may have **extremely steep regions**.
- Repeated weight multiplication causes **exploding gradients**.
- Large updates may throw parameters far from good regions.
- **Example:**
  Deep linear network where output = $W \times W \times W \times x$


Loss Function for some Machine Learning Model
Higher Loss
Lower Loss
P1
P2
P3
f(x)
x
P1 : Local Min
P2 : Saddle Point
P3 : Global Min

Stochastic Gradient Descent (SGD)

- The **learning rate (ϵ)** is a very important parameter for SGD.
- The learning rate **ϵ should be reduced after each epoch** in general.
- This is because random sampling of batches acts as a **source of noise**.
- Due to this noise, SGD may keep **oscillating around the minima** without actually reaching it.
- This oscillatory behavior is illustrated in **Fig. 19 (Stochastic Gradient Descent)**.
- **Example:**
  With mini-batches, each update sees only part of the data, causing noisy updates.

SGD vs Batch Gradient Descent (BGD)

- The true gradient of the total cost function (over the entire dataset) becomes **zero at the minimum.**
- Hence, **Batch Gradient Descent (BGD)** can use a **fixed learning rate.**
- SGD uses an approximate gradient, hence learning rate scheduling is required.

Convergence Conditions for SGD (Convex Case)

- Under convexity assumptions, convergence of SGD is guaranteed if:
  - The sum of learning rates diverges
  - The sum of squared learning rates converges

Choosing the Learning Rate

- Setting the learning rate **too low**:
  - Training proceeds **very slowly**
  - Algorithm may get stuck at a **high cost value**
- Setting the learning rate **too high**:
  - Causes **large oscillations**
  - May push learning **outside the optimal region**
- Best practice:
  - Monitor the **first several iterations**
  - Set learning rate **higher than the best-performing one**, but not too high to cause instability

Advantages of SGD

- Time taken to compute a weight update **does not grow** with number of training examples.
- Each update is computed using a **batch of samples**, independent of dataset size.
- Suitable for **very large datasets**.

Momentum

- Momentum accumulates an **exponentially decaying moving average of past gradients**.
- This accumulated quantity is called **velocity**.
- Momentum = mass × velocity (unit mass assumed).
- The velocity determines the **direction of the next step**.

Momentum Weight Update

- Step size now depends on:
  - How **large** the gradients are
  - How **aligned** the gradients are over iterations
- If gradients keep pointing in the same direction:
  - Velocity keeps increasing
  - Step size increases
- At terminal velocity:
  - Step size becomes
    $\epsilon \, ||g|| \, / \, (1 - \alpha)$

Momentum Hyperparameter (α)

- α controls how much past gradients influence current update
- Common values:
  - α = 0.5
  - α = 0.9
  - α = 0.99
- Using α = 0.9 makes the speed **10 times faster**
- **Fig. 21:** Momentum behavior

Problems Solved by Momentum

- Momentum aims to solve:
- **Poor conditioning of the Hessian matrix**
- **Variance in the stochastic gradient**

Momentum and Poor Conditioning

- Poorly conditioned Hessian produces:
  - Long, narrow valleys
  - Steep sides
- Gradient Descent:
  - Oscillates across the valley
  - Wastes steps
- Momentum:
  - Moves smoothly along the **length of the valley**
  - Reduces oscillations
- **Fig. 21 Explanation:**
  Momentum cuts across contours efficiently, unlike standard GD.

Newtonian Interpretation of Momentum

- Momentum can be viewed as **Newtonian dynamics** of a particle sliding down a hill.
- It involves solving **differential equations** numerically.
- Two forces act on the particle.

Nesterov Momentum

- Nesterov Momentum is a **slight modification** of standard momentum.
- Gradient is computed **after applying current velocity**.
- This acts as a **correction factor**.

Nesterov Momentum Weight Update

- Instead of computing gradient at θ:
  - Compute gradient at $(\theta + \alpha v)$
- Accounts for the **future position** of parameters

Intuition Behind Nesterov Momentum

- At point θ, momentum will shift parameters by **αv**
- The next position will be near **(θ + αv)**
- Hence, it is better to:
  - Look ahead
  - Compute gradient from that future position
- Leads to:
  - Better anticipation
  - Faster convergence
  - Reduced overshooting